
Week-6

1.1 Defining Data: DDL

Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database. DDL is a set of SQL commands used to create, modify, and delete database structures but not data..

SQLs data definition language (DDL) defines information about each relation in a database which is listed below in detail:

- DDL specifies the schema of each relation i.e. the logical design of each relation which states the name of that relation, attributes it carries and also specifies the domain of those attributes.
- DDL specifies the integrity constraint which makes sure that any changes made to the database don't affect the consistency of data.
- DDL also maintains the set of indices for each relation which let you retrieve the records from the database quickly.
- DDL maintains the information about the security of data in the database and it also keeps the information regarding the authorization for each relation in the database.
- DDL also describes the storage structure of each relation on the hard disk.

List of DDL commands:

- **CREATE:** This command is used to create the database or its objects (like table, index, function, views, store procedure, and triggers).
- **DROP:** This command is used to delete objects from the database.
- **ALTER:** This is used to alter the structure of the database.
- **TRUNCATE:** This is used to remove all records from a table, including all spaces allocated for the records are removed.
- **COMMENT:** This is used to add comments to the data dictionary.
- **RENAME:** This is used to rename an object existing in the database.

1.2 CREATE:

There are two CREATE statements available in SQL:

1. CREATE DATABASE
-

2. CREATE TABLE

The **CREATE DATABASE** statement is used to create a new database in SQL.

Syntax:

```
CREATE DATABASE database_name;
```

For Example:

```
CREATE DATABASE TEST;
```

This query will create a new database in SQL and name the database as *TEST*

The CREATE TABLE statement is used to create a table in SQL. We know that a table comprises of rows and columns. So while creating tables we have to provide all the information to SQL about the names of the columns, type of data to be stored in columns, size of the data etc.

Syntax:

```
CREATE TABLE table_name
(
column1 data_type(size),
column2 data_type(size),
column3 data_type(size),
....
);
```

Where

table_name: name of the table.

column1 :name of the first column.

data_type: Type of data we want to store in the particular column.

For example, **integer** for integer data.

size: Size of the data we can store in a particular column. For example if for a column we specify the data_type as int and size as 10 then this column can store an integer Number of maximum 10 digits.

Example :

```
CREATE TABLE Student
(
ROLL_NO integer,
NAME varchar(20),
address varchar(20)
);
```

INDEX

Indexes are **special lookup tables that the database search engine can use to speed up data retrieval**. Simply put, an index is a pointer to data in a table. An index in a database is very similar to an index in the back of a book.

PostgreSQL provides several index types: B-tree, Hash, GiST, SP-GiST, GIN and BRIN. Each index type uses a different algorithm that is best suited to different types of queries. By default, the CREATE INDEX command creates B-tree indexes, which fit the most common situations.

Syntax:

```
CREATE INDEX name ON table USING HASH (column);
```

For Example:

```
CREATE INDEX rollno_index on student using btree(rollnum);
```

1.3 ALTER

ALTER TABLE is used to add, delete/drop or modify columns in the existing table. It is also used to add and drop various constraints on the existing table.

ALTER TABLE – ADD

ADD is used to add columns into the existing table. Sometimes we may require adding additional information, in that case we do not require to create the whole database again, **ADD** comes to our rescue.

Syntax:

```
ALTER TABLE table_name ADD COLUMN Column_name Datatype(SIZE);
```

For Example:

```
ALTER TABLE STUDENT ADD COLUMN Phoneno integer;
```

ADD CONSTRAINT:

The syntax for creating a unique constraint using an ALTER TABLE statement in PostgreSQL is:

```
ALTER TABLE table_name ADD CONSTRAINT constraint_name  
UNIQUE (column1, column2, ... column_n);
```

For Example:

```
ALTER TABLE STUDENT ADD CONSTRAINT phone_unique UNIQUE(Phoneno);  
ALTER TABLE STUDENT ADD CONSTRAINT roll_pk PRIMARY  
KEY(ROLL_No);
```

ALTER TABLE – DROP

DROP COLUMN is used to drop column in a table. Deleting the unwanted columns from the table.

Syntax:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

For Example:

```
ALTER TABLE STUDENT DROP COLUMN BRANCH;
```

DROP CONSTRAINT:

The syntax for dropping constraint using an ALTER TABLE statement in PostgreSQL is

```
ALTER TABLE table_name DROP CONSTRAINT constraint_name;
```

For Example;

```
ALTER TABLE STUDENT DROP CONSTRAINT roll_pk;
```

- Change the name of column NAME to FIRST_NAME in table Student

```
ALTER TABLE Student RENAME COLUMN NAME TO FIRST_NAME
```
- Change the name of the table Student to Student_Details

```
ALTER TABLE Student RENAME TO Student_Details;
```

1.4 DROP

DROP is schema changing statement in SQL

DROP TABLE:

DROP TABLE removes tables from the database. Only its owner may destroy a table. To empty a table of rows without destroying the table, use DELETE or TRUNCATE.

DROP TABLE always removes any indexes, rules, triggers, and constraints that exist for the target table. However, to drop a table that is referenced by a view or a foreign-key constraint of another table, CASCADE must be specified. (CASCADE will remove a dependent view entirely, but in the foreign-key case it will only remove the foreign-key constraint, not the other table entirely.

Parameters

IF EXISTS

Do not throw an error if the table does not exist. A notice is issued in this case.

Name

The name (optionally schema-qualified) of the table to drop.

CASCADE

Automatically drop objects that depend on the table (such as views).

RESTRICT

Refuse to drop the table if any objects depend on it. This is the default.

For Example:

```
1) DROP TABLE IF EXISTS films;
```

This does not throw an error.

```
2) DROP TABLE STUDENT CASCADE;
```

Automatically drop objects that depend on the table **student** (such as views)

```
3) DROP TABLE STUDENT RESTRICT;
```

This does not drop the table student since child table referencing STUDENT may exist.

1.5 Temporary tables

A temporary table, as its name implied, is a short-lived table that exists for the duration of a database session. PostgreSQL automatically drops the temporary tables at the end of a session or a transaction.

To create a temporary table, you use the CREATE TEMPORARY TABLE statement.

Syntax:

```
CREATE TEMPORARY TABLE temp_table_name(  
    column_list    );
```

In this syntax:

- First, specify the name of the temporary table after the CREATE TEMPORARY TABLE keywords.
- Second, specify the column list, which is the same as the one in the CREATE TABLE statement.

The TEMP and TEMPORARY keywords are equivalent so you can use them interchangeably

Removing a PostgreSQL temporary table

- To drop a temporary table, you use the DROP TABLE statement. The following statement illustrates how to drop a temporary table:

```
DROP TABLE temp_table_name;
```

- Unlike the CREATE TABLE statement, the DROP TABLE statement does not have the TEMP or TEMPORARY keyword created specifically for temporary tables.

There are 2 types of Temporary Tables: Local Temporary Table, and Global Temporary Table. These are explained as following below.

1. **Local Temporary Table:**

A Local Temp Table is available only for the session that has created it. It is automatically dropped (deleted) when the connection that has created it, is closed.

2. **Global Temporary Table:**

3. The rows registered into a global temporary table are private to the session that inserts them, which means that they are only visible to that session but not any other sessions. These rows can be preserved for the session time or only for the current transaction. This is defined at temporary table creation time using the clauses - **ON COMMIT PRESERVE ROWS** or **ON COMMIT DELETE ROWS..**

Example:

```
CREATE GLOBAL TEMPORARY TABLE a_gtt_table (  
id integer PRIMARY KEY,  
lbl text NOT NULL,  
creation_date timestamp CHECK (creation_date > '2021-01-01')  
) ON COMMIT PRESERVE ROWS;
```

1.6 External Tables:

CREATE FOREIGN TABLE

CREATE FOREIGN TABLE creates a new foreign table in the current database. The table will be owned by the user issuing the command.

If a schema name is given (for example, CREATE FOREIGN TABLE myschema.mytable ...) then the table is created in the specified schema. Otherwise it is created in the current schema. The name of the foreign table must be distinct from the name of any other foreign table, table, sequence, index, view, or materialized view in the same schema.

CREATE FOREIGN TABLE also automatically creates a data type that represents the composite type corresponding to one row of the foreign table. Therefore, foreign tables cannot have the same name as any existing data type in the same schema.

If PARTITION OF clause is specified then the table is created as a partition of parent_table with specified bounds.

To be able to create a foreign table, you must have USAGE privilege on the foreign server, as well as USAGE privilege on all column types used in the table.

Examples

Create foreign table films, which will be accessed through the server film_server:

```
CREATE FOREIGN TABLE films (  
  code      char(5) NOT NULL,  
  title     varchar(40) NOT NULL,  
  did       integer NOT NULL,  
  date_prod date,  
  kind      varchar(10),  
  len       interval hour to minute  
)  
SERVER film_server;
```

Create foreign table `measurement_y2016m07`, which will be accessed through the server `server_07`, as a partition of the range partitioned table `measurement`:

```
CREATE FOREIGN TABLE measurement_y2016m07  
  PARTITION OF measurement FOR VALUES FROM ('2016-07-01') TO ('2016-  
08-01')  
  SERVER server 07;
```

1.7 Managing constraints

PostgreSQL - CONSTRAINTS

1. NOT NULL Constraint – Ensures that a column cannot have NULL value.
2. UNIQUE Constraint – Ensures that all values in a column are different.
3. PRIMARY Key – uniquely identifies each row/record in a database table.
4. FOREIGN Key – Constrains data based on columns in other tables.
5. CHECK Constraint – The CHECK constraint ensures that all values in a column satisfy certain conditions

For Example:

```
CREATE TABLE EMPLOYEE  
(SSN INTEGER PRIMARY KEY,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INTEGER,  
  PHONE INTEGER UNIQUE,  
  CHECK (age>18));
```

```

postgres=# \d employee
Table "public.employee"
Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
ssn    | integer                |           | not null |
name   | character varying(20)  |           | not null |
age    | integer                |           |          |
phone  | integer                |           |          |
Indexes:
    "employee_pkey" PRIMARY KEY, btree (ssn)
    "employee_phone_key" UNIQUE CONSTRAINT, btree (phone)
Check constraints:
    "employee_age_check" CHECK (age > 18)

```

CREATING FOREIGN Key Constraints:

```

CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);
CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    product_no integer REFERENCES products (product_no),
    quantity integer
);

```

ADD CONSTRAINT:

The syntax for creating a unique constraint using an ALTER TABLE statement in PostgreSQL is:

```

ALTER TABLE table_name ADD CONSTRAINT constraint_name
UNIQUE (column1, column2, ... column_n);

```

For Example:

```

ALTER TABLE STUDENT ADD CONSTRAINT phone_unique UNIQUE(Phoneno);
ALTER TABLE STUDENT ADD CONSTRAINT roll_pk PRIMARY
KEY(ROLL_No);

```

DROP CONSTRAINT:

The syntax for dropping constraint using an ALTER TABLE statement in PostgreSQL is

```

ALTER TABLE table_name DROP CONSTRAINT constraint_name;

```